

Software Project – I
College Social Media

BY
Aswin A
BP231033

Under the Guidance of
Asst.Prof. DR. S. Anthony Philomen Raj M.C.A, Ph. D.



PG DEPARTMENT OF COMPUTER APPLICATION
SACRED HEART COLLEGE (AUTONOMOUS)
TIRUPATTUR-635 601

October 2024

Sacred Heart College (Autonomous)
(Affiliated to Thiruvalluvar University)
Accredited by NAAC (5th Cycle) with 'A++' Grade [3.31]
Tirupattur District – 635 601
PG AND RESEARCH DEPARTMENT OF
COMPUTER APPLICATIONS

Reg. No: BP231033	Class: II-MCA
Project Title: College Social media	

Certified to be a Bonafide Project work done by Aswin A in MCA366J
Software Project - I during the odd semester July – October 2024.

Head of the Department

Guide Signature

Submitted for Viva Voce Examination held on..... at
Sacred Heart College (Autonomous), Tirupattur District- 635 601

External 1

External II

Contents

S.NO	DESCRIPTION	PAGE NO
1.	Abstract Introduction 1.1 Purpose 1.2 System Overview and Development Goal 1.3 Problem Statement 1.4 System Model 1.5 Project Time Line Diagram (Gant Chart)	1-5
2.	2.1 Module Identification and Description 2.2 User Role and Characteristic 2.3 Functional Requirement 2.4 Performance Requirement 2.5 Hardware and Software details (Implementation and Deployment Side) 2.6 Conceptual Diagram (Database ER Diagram)	6-14
3.	3.1 System Architecture Diagram 3.2 UML Design Models 3.2.1 Use Case Diagram 3.2.2 Class Diagram 3.2.3 Activity Diagram 3.2.4 Sequence Diagram 3.4 Database Design Models 3.4.1 Table Structure 3.4.2 Table Relationship Diagram 3.5 UI Design	15-31

4.	4.1 Implementation of Java Code	32-43
5.	5.1 Test Plan 5.2 Test Cases 5.3 Test Report	44-53
6.	6.1 Implementation and Deployment	54-55
7.	7.1 I/O Report 7.2 Conclusion 7.3. Reference	56-60

Abstract:

The **SHC Social Media App** is a mobile-based platform designed to improve communication within the college community by offering a centralized space for sharing official announcements and updates. This app primarily serves two user groups: administrators and students. Administrators (such as faculty or management) have the ability to upload posts containing text, images, or videos that may include important updates like event announcements, policy changes, or other management-related information. Additionally, they can delete posts when necessary, maintaining the relevance and accuracy of the content.

The posts are categorized by Management, and the app dynamically fetches the department names from Firebase. Firebase is also used to store and retrieve post content efficiently. On the student side, users can view these posts, stay updated with college-related information, and interact by liking the posts. However, students cannot upload or delete posts, ensuring that the flow of information remains official and controlled.

This platform enhances the flow of communication between the college administration and students, making critical information easily accessible at any time. With a simple user interface and real-time updates via Firebase, the app ensures that students remain informed about important happenings within the institution, contributing to a more organized and well-informed campus environment. The like feature also allows the administration to gauge student engagement with different announcements.

Introduction:

In today's academic environment, digital communication plays a crucial role in bridging the gap between College Management and students. Recognizing this need, the **SHC Social App** was conceived as a solution to enhance interaction and foster engagement within **Sacred Heart College**. The app aims to provide a structured, yet dynamic platform where administrators can share important updates, events, and media with students in an interactive way.

The app offers two primary modules:

- **Admin Module:** College Management can log in and create posts consisting of images, videos, and descriptions. Each post is automatically displayed at the top of the feed, with the Relevant Department name. Admins also have full control over the content they upload, including the ability to edit, delete, or update posts.
- **User Module:** Students can log in to view posts shared by department admins. They can like posts, and these likes are stored in the database along with their username, enabling admins to see a list of who engaged with their content. This creates an interactive environment where students feel connected to their department's activities and announcements.

The **SHC Social App** was developed with the goal of fostering a more connected and engaged college community. By replicating familiar social media features in a controlled, educational environment, the app offers both administrators and students an accessible platform for communication and interaction.

CHAPTER – I

1.1 Purpose:

The purpose of the SHC Social Media App is to provide a streamlined and efficient communication channel that facilitates the dissemination of crucial information from the SHC Management to students. In an academic environment where timely access to information is vital, the app aims to centralize and simplify the sharing of announcements, events, and management-related updates. By consolidating communication within a single platform, the app addresses the challenges posed by traditional methods, such as email, notice boards, or departmental messages, which can often lead to missed announcements or information overload.

The application is designed with user experience in mind, allowing the admin (SHC Management) to easily upload and manage posts, ensuring that students have immediate access to the latest updates. The interactive nature of the platform enables students to engage with the content by liking posts, fostering a sense of community and connection within the student body.

Moreover, the SHC Social Media App aims to reduce the fragmentation of communication channels, allowing students to focus on one primary source for all their informational needs. This enhances not only the visibility of important announcements but also the accountability of the management in keeping students informed.

1.2 Objectives:

The purpose of the SHC Social Media App is to streamline communication of important information from the SHC Management to the students in an efficient and organized manner. The platform is designed to simplify the sharing of announcements, events, and management-related updates in a centralized location accessible to all students. By providing a user-friendly interface, the app allows SHC Management to easily upload and delete posts, ensuring that the most relevant information is readily available. Students can view, like, and interact with these posts, promoting engagement and fostering a sense of community within the college environment. This app aims to enhance the effectiveness of communication channels, ensuring timely and consistent delivery of information while reducing reliance on multiple fragmented communication methods. Additionally, the platform emphasizes security and scalability, allowing it to grow alongside the needs of the institution.

The intuitive design also facilitates easy navigation, helping students quickly find and access important updates, ultimately contributing to a more informed student body.

1.3 Problem Statement:

Effective communication within academic institutions, especially between management and students, can be challenging due to fragmented communication channels such as email, notice boards, or individual department messaging systems. This lack of a centralized platform often results in important announcements being missed or delayed.

Proposed Solution:

The SHC Social Media App proposes a single, centralized platform where the SHC Management can upload posts, such as announcements, events, or updates, which are instantly visible to all students. Students can view, like, and interact with the content. The admin retains control over uploading and deleting posts, maintaining the flow of critical information. The app is designed to be lightweight, with no involvement of specific departments or logos, further simplifying the process for both admin and students.

1.4 System Model:

The **SHC Social App** follows a client-server architecture, where the **Android app** acts as the client, and the **Realtime Firebase** as the backend, managing all interactions and data.

1.4.1 User Roles and Interfaces:

- **Admin (SHC Management):**
 - Upload posts (images, videos, text).
 - Delete posts when necessary.
 - Moderate the content visible to students.
- **Student Users:**
 - View the posts uploaded by the admin.
 - Like the posts to interact with the content.
 - Access all posts without the ability to modify or delete.

Basic Data Flow:

1. Admins log in -> Upload posts -> Delete posts -> Posts are saved or deleted from firebase.
2. Students log in -> View posts -> Like posts -> Likes are stored in the database.

1.4.2 System Architecture:

The SHC Social Media App follows a client-server architecture, with Firebase serving as the backend for real-time data storage, media handling, and authentication. The app's frontend is built for Android devices using Java and XML, providing a simple interface for both admins and students. Firebase Database is used to store posts, while Firebase Storage handles media files (images, videos). Authentication ensures that only the admin can upload and delete posts, while students can only view and like the posts.

- **Client Side:** Android application (developed in Java and XML) for both students and admins.
- **Backend:** Firebase for real-time database services, Firebase Storage for media storage, and Firebase Authentication for secure admin login.

1.5 Project Time Line Diagram (Gantt Chart):

Below is an outline of a Gantt chart for the development of the **SHC Social App**. This chart visualizes the major phases of the project, from requirement gathering to testing and deployment.

Task	W 1	W 2	W 3	W 4	W 5	W 6	W 7	W 8	W 9	W 10	W 11	W 12	W 13	W 14	W 15
Planning & Research															
System Design & Architecture															
UI/UX Design															
Student Interface Development															
Admin Interface Development															
Backend System Setup															
API Development															
Testing															
Final Deployment & Launch															

CHAPTER II

2.1 Module Identification and Description:

2.1.1 Admin Module:

- **Description:** This module is designed for admins who can create, manage, and track posts. Admins are responsible for uploading media content (images, videos) with descriptions and tracking user engagement through likes.
- **Features:**
 - **Login:** Admins use predefined credentials to log in.
 - **Upload Post:** Admins can upload posts with an image and description. Posts are displayed with the admin's profile image and name.
 - **CRUD Operations:** Admins can create, update, delete, and read their posts.

Upload Post Module:

The Upload Post module is a critical component of the SHC Social Media App, designed specifically for the SHC Management to share important announcements, events, and updates with students. This module operates as follows:

1. **User Authentication:** Admins log in using predefined credentials, ensuring that only authorized personnel can access the upload functionality.
2. **Post Creation:** Once logged in, the admin can initiate the post creation process. This includes selecting an image and entering a description that encapsulates the content of the post.
3. **User Interface:** The interface for uploading posts is intuitive, allowing the admin to easily navigate through the required steps. The admin can preview the image before uploading, ensuring that the selected content is appropriate.
4. **Data Handling:** Upon submission, the post data, including the image and description, is stored in the Firebase database. The image is uploaded to Firebase Storage, while the post details are saved in the Realtime Database.
5. **Post Visibility:** Once successfully uploaded, the post is immediately visible to all students. Each post displays the admin's profile image and name, lending credibility and transparency to the information being shared.

6. **User Interaction:** Students can view these posts and interact with them by liking, which helps gauge engagement and interest in the shared content.

Delete Post Module:

The Delete Post module allows the SHC Management to maintain control over the content shared within the app. It ensures that outdated or irrelevant information can be removed quickly and efficiently. This module operates as follows:

1. **Post Management:** Admins have access to all the posts they have created. The interface allows for easy navigation through existing posts, which can be sorted or filtered based on criteria such as date or content type.
2. **Deletion Process:** When an admin decides to delete a post, they can select the specific post from the list. A confirmation dialog appears to prevent accidental deletions, prompting the admin to confirm the action.
3. **Data Update:** Upon confirmation, the selected post is removed from the Firebase database, ensuring that it is no longer visible to students. This update is reflected in real-time, allowing for immediate content management.
4. **User Feedback:** The app provides feedback to the admin upon successful deletion, often through a toast message confirming that the post has been successfully removed.
5. **Maintaining Relevance:** By enabling the deletion of posts, the app ensures that only relevant and current information is available to students, enhancing the overall communication effectiveness.

2.1.2 User Module:

- **Description:** This module is for students (users) who interact with the app by viewing and liking posts.
- **Features:**
 - **Login:** Students use their assigned credentials to log in.
 - **View Posts:** Students can view the latest posts uploaded by admins, with the most recent posts displayed first.
 - **Like Posts:** Students can like posts, and these likes are recorded in the firebase database.

Login Module

The Login module is the entry point for students into the SHC Social Media App. It ensures that access to the platform is secure and restricted to authorized users. The functionality includes:

1. **User Authentication:** Students use their assigned credentials to log in. This may involve entering a username and password, which are validated against the database to confirm the user's identity.
2. **Session Management:** Upon successful login, the app creates a user session, allowing students to navigate the app without needing to log in repeatedly. If login fails, appropriate error messages are displayed to help the user troubleshoot.
3. **User Interface:** The login interface is designed to be user-friendly, with clear prompts for entering credentials and a straightforward process for logging in.
4. **Security Features:** The module may also include features such as password recovery options and security measures to protect user data, ensuring a safe experience for students.

2.2 User Role and Characteristics:

2.2.1 Admin:

- **Role:** Admin responsible for managing posts and viewing student interactions.
- **Characteristics:**
 - Has access to upload and manage media posts.
 - Can view and track which students liked their posts.
 - Can perform CRUD operations on posts.
 - Has a unique login credential with higher privileges compared to regular users.

2.2.2 User (Student):

- **Role:** Students who can view and interact with the content uploaded by admins.
- **Characteristics:**
 - Can view posts uploaded by admins.
 - Can like posts to show engagement.
 - Has a unique login credential but with limited access (view and like features).

User Role	Characteristics
Admin (SHC Management)	- Manages the platform, posts, and content. - Uploads important announcements, events, and updates. - Deletes or updates posts as necessary. - Has full CRUD (Create, Read, Update, Delete) privileges over all posts. - Uses predefined credentials to log in. - Ensures timely and accurate information dissemination.
Student (User)	- Views posts uploaded by the admin. - Can like posts to show engagement. - Can view posts in a chronological order, with the most recent posts first. - Has read-only access; no post creation, deletion, or editing rights. - Uses assigned credentials to log in and access content. - Interacts with the posts by liking them, but cannot upload content.

2.3 Functional Requirements:

The SHC Social App must meet the following functional requirements:

1. User Authentication:

- The system should authenticate both admins and users with their respective credentials.
- Admin credentials Sample: username: shcmanagement, password: shcmanagement@edu.
- User credentials Sample: username: AU200517, password: 02/10/2003.

2. Admin Module:

- Admins should be able to log in and access the admin dashboard.
- Admins should be able to upload posts with an image and description.
- Admins should be able to view a list of students who liked their posts.
- Admins should be able to manage their profile and perform CRUD operations on their posts.

3. User Module:

- Users should be able to log in and access the user dashboard.
- Users should be able to view posts uploaded by admins.
- Users should be able to like posts, and the like should be stored in the database.

4. Database Management:

- Store all posts and likes in a structured way so that they can be retrieved and displayed efficiently.
- Ensure that admin and user data is securely stored.

5. Post Display:

- Posts should be displayed in a feed format with the most recent posts shown at the top.
- Each post should display the admin's profile image and name.

2.4 Performance Requirements for the SHC Social Media App

The performance requirements for the SHC Social Media App are essential to ensure a smooth, responsive, and scalable user experience for both admins and students. The following performance metrics and requirements must be met:

1. Response Time:

- The app should load within 3 seconds on most devices with an average internet connection.
- Viewing posts should be instantaneous, with no more than 2 seconds delay in displaying new or existing posts after a request is made.
- Liking a post should reflect immediately, and the like count should update in real-time without reloading the entire page.

2. Concurrent Users:

- The app should support multiple concurrent users, with at least 500 students able to access the platform at the same time without performance degradation.
- Firebase Realtime Database should handle up to 1,000 read and write operations per second to accommodate student interactions, such as viewing and liking posts.

3. Scalability:

- The app's backend infrastructure should be scalable to handle increasing numbers of students and posts as the college grows. This includes seamless scaling of the Firebase Database and Storage.
- Admin actions, such as uploading or deleting posts, should not affect the performance for student users during peak traffic.

4. Storage Efficiency:

- Images or videos uploaded by the admin should be optimized for mobile viewing to reduce bandwidth usage. Images should load in less than 2 seconds under normal network conditions.
- Firebase Storage should be configured to store and deliver media efficiently, ensuring minimal load times for post images or videos.

5. Data Synchronization:

- All posts and updates should synchronize across devices in real-time. For example, when a post is uploaded or deleted, it should appear/disappear across all students' devices instantly.
- Liked posts should update in real-time across all user interfaces, ensuring consistency of interaction data.

6. System Uptime:

- The app should have an uptime of 99.9% to ensure reliability. Downtime, especially during critical periods like exam schedules or important college events, must be avoided.

7. Battery and Resource Usage:

- The app must be optimized to consume minimal battery and device resources. The loading of images, posts, and user interactions should be handled efficiently to avoid excessive CPU or memory usage on the user's device.

8. Security Performance:

- The app must ensure all user interactions and data are handled securely, with minimal delay in authentication (less than 2 seconds to log in).
- Any sensitive operations, such as post deletions by the admin, must be logged securely and should not introduce performance bottlenecks.

2.5 Hardware and Software Details (Implementation and Deployment Side):

1. Hardware Requirements:

- **Processor:** Intel Core i3.
- **RAM:** 8GB.
- **Storage:** 20GB free disk space.
- **Display:** 1366x768 resolution or higher.
- **Internet Connection:** Required for testing and backend API requests.

2. Software Requirements:

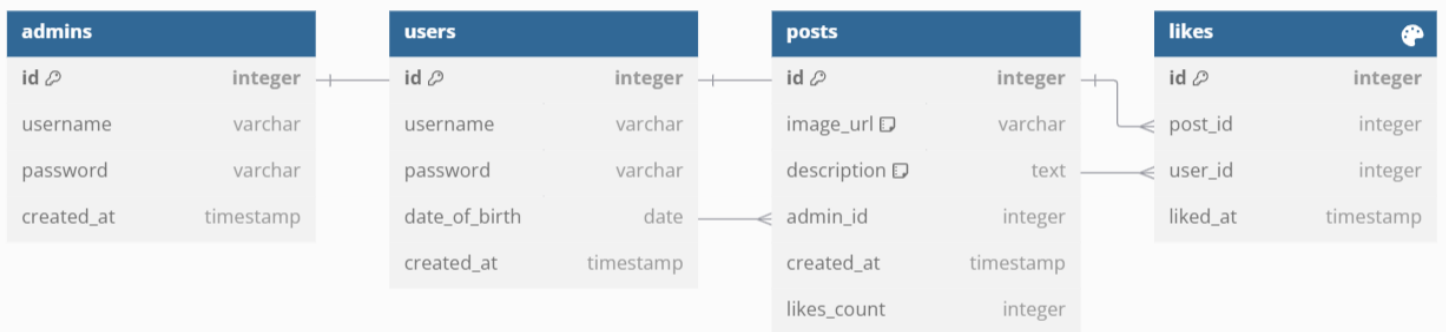
Development Side:

- **Operating System:** Windows 10/11, macOS, or Linux.
- **IDE:** Android Studio (latest version) for frontend development.
- **Backend:** Firebase realtime Database and Storage.
- **Programming Languages:**
 - **Frontend:** Java (Android)
 - **Backend:** Firebase Nodes and JSON.
 - **Database Management:** Firebase realtime database

Deployment Side:

- **Deployment Environment:** realtime Database for initial testing and deployment, with the possibility of scaling to a cloud-hosted server (AWS, Google Cloud) for production.

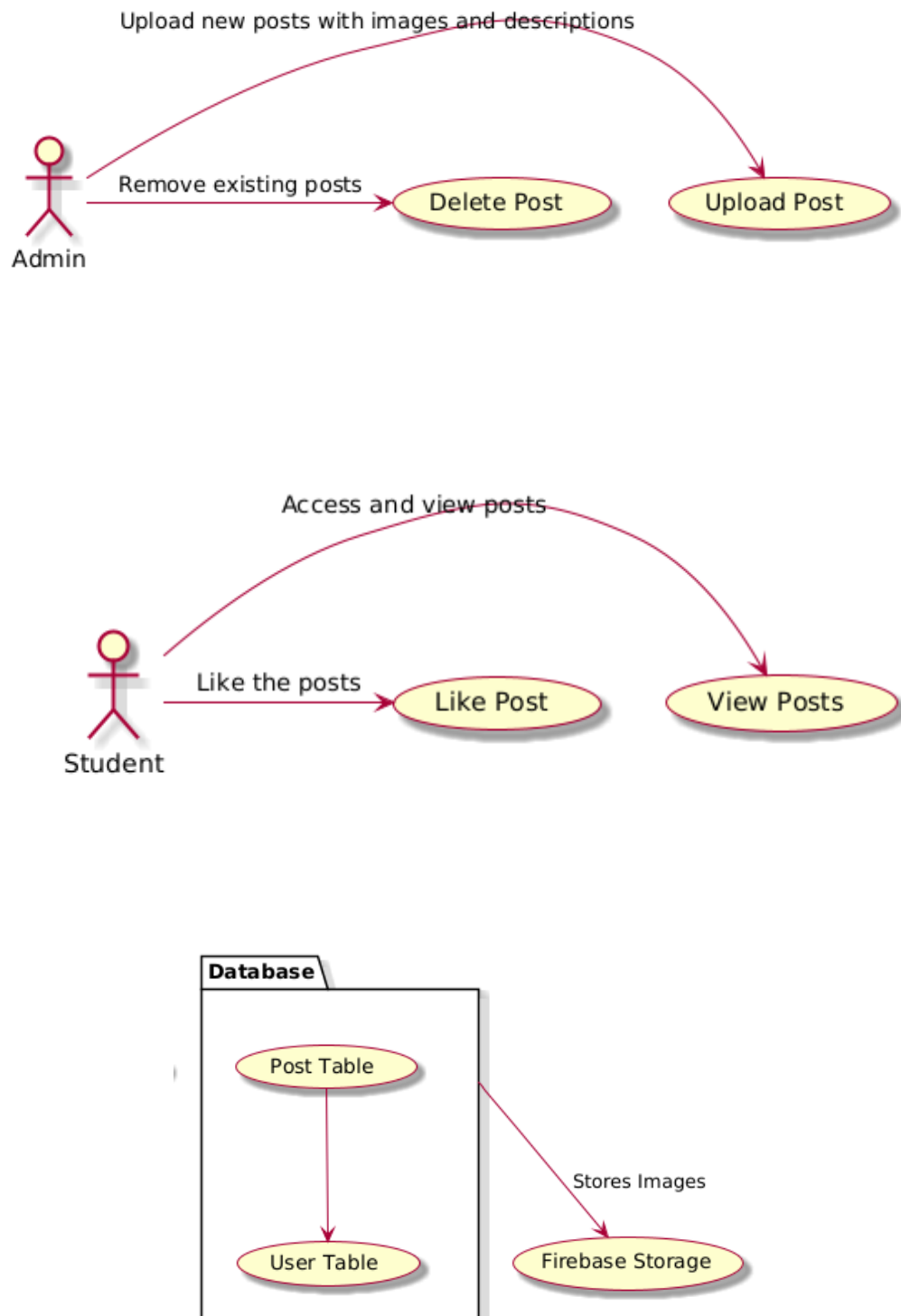
2.6 Conceptual Diagram (DB Entity Relationship Diagram):



CHAPTER – III

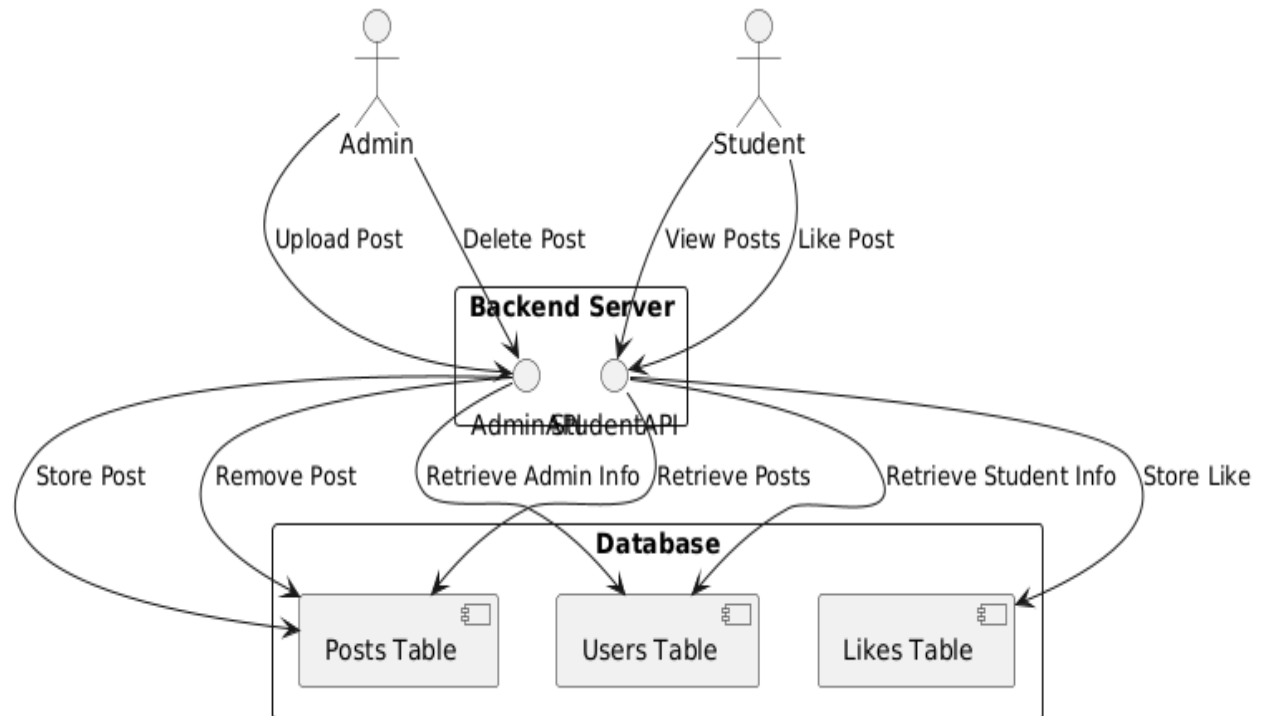
DESIGN SPECIFICATIONS

3.1 System Architecture Diagram:

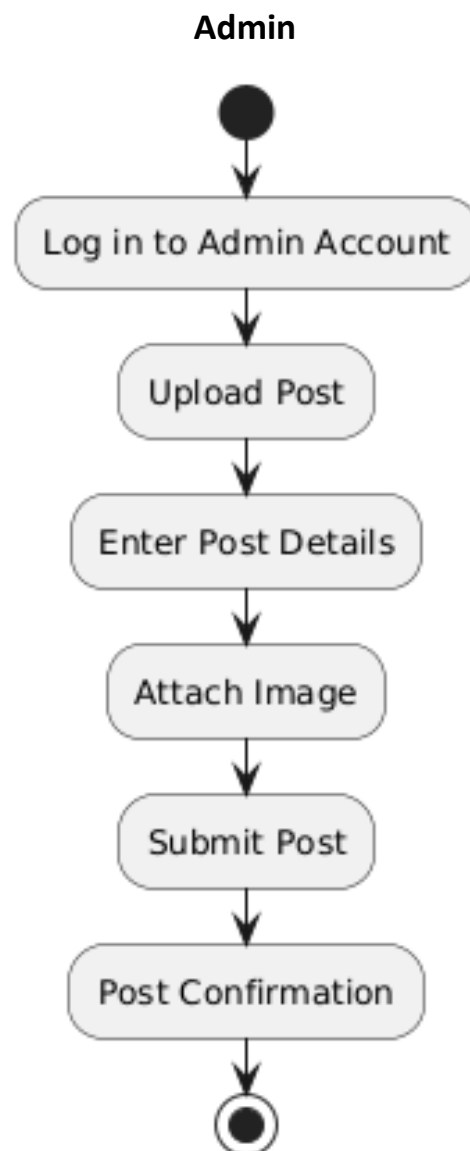


3.2 UML Design Models

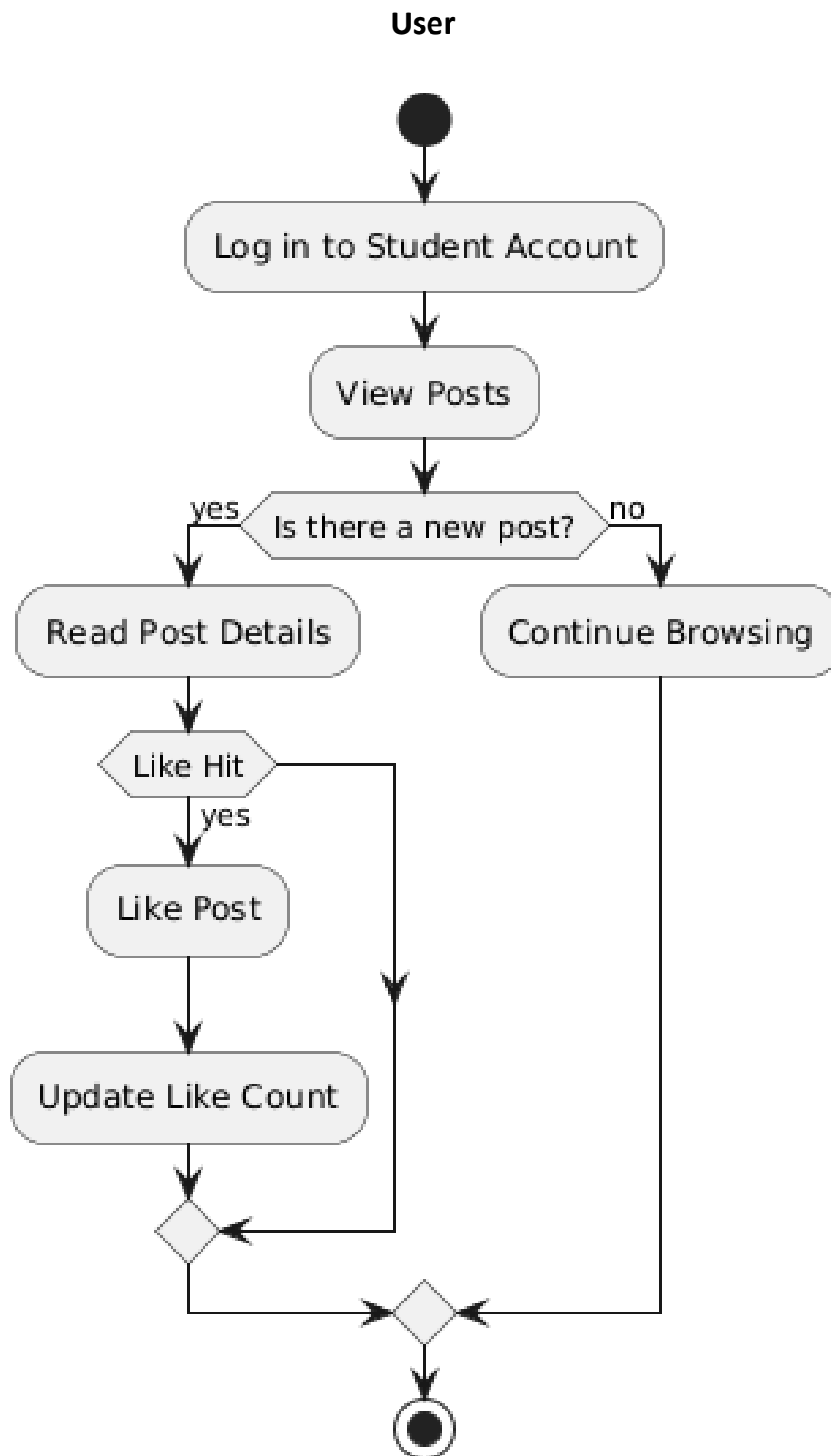
Use Case Diagram:



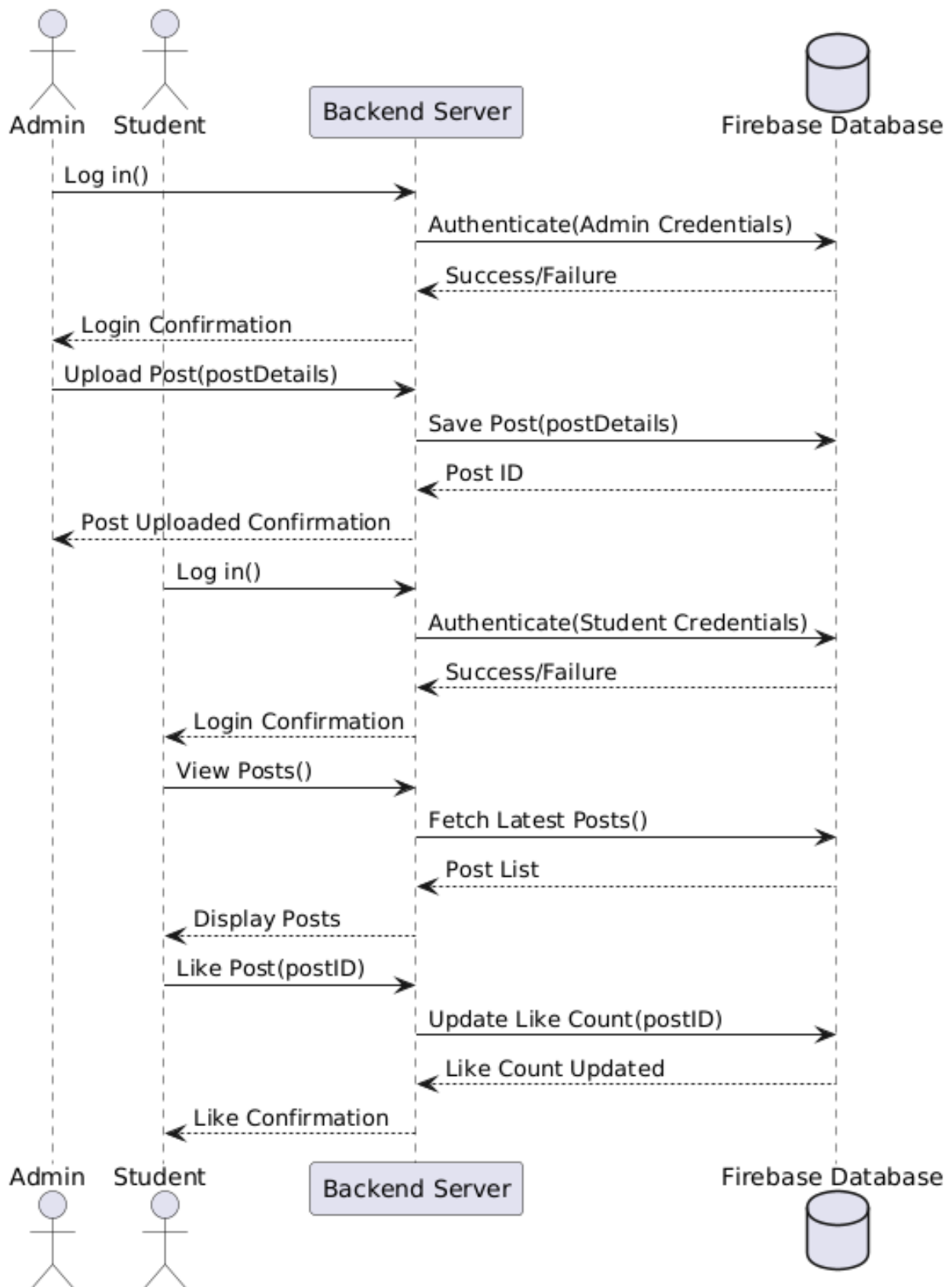
Activity Diagram: Admin



Activity Diagram: User



Sequence Diagram:



3.3 Database Design

S.No	TABLE NAME	DESCRIPTION
1.	User	This table stores user information for both students and staff, including Name, User ID, Department, and Password.
2.	Post	This table contains the posts uploaded by the admin, including Post ID, Image URL, Description, Upload Date, and Author ID.
3.	Likes	This table tracks the likes on posts, recording the Post ID, User ID (student), and the Timestamp of the like action.

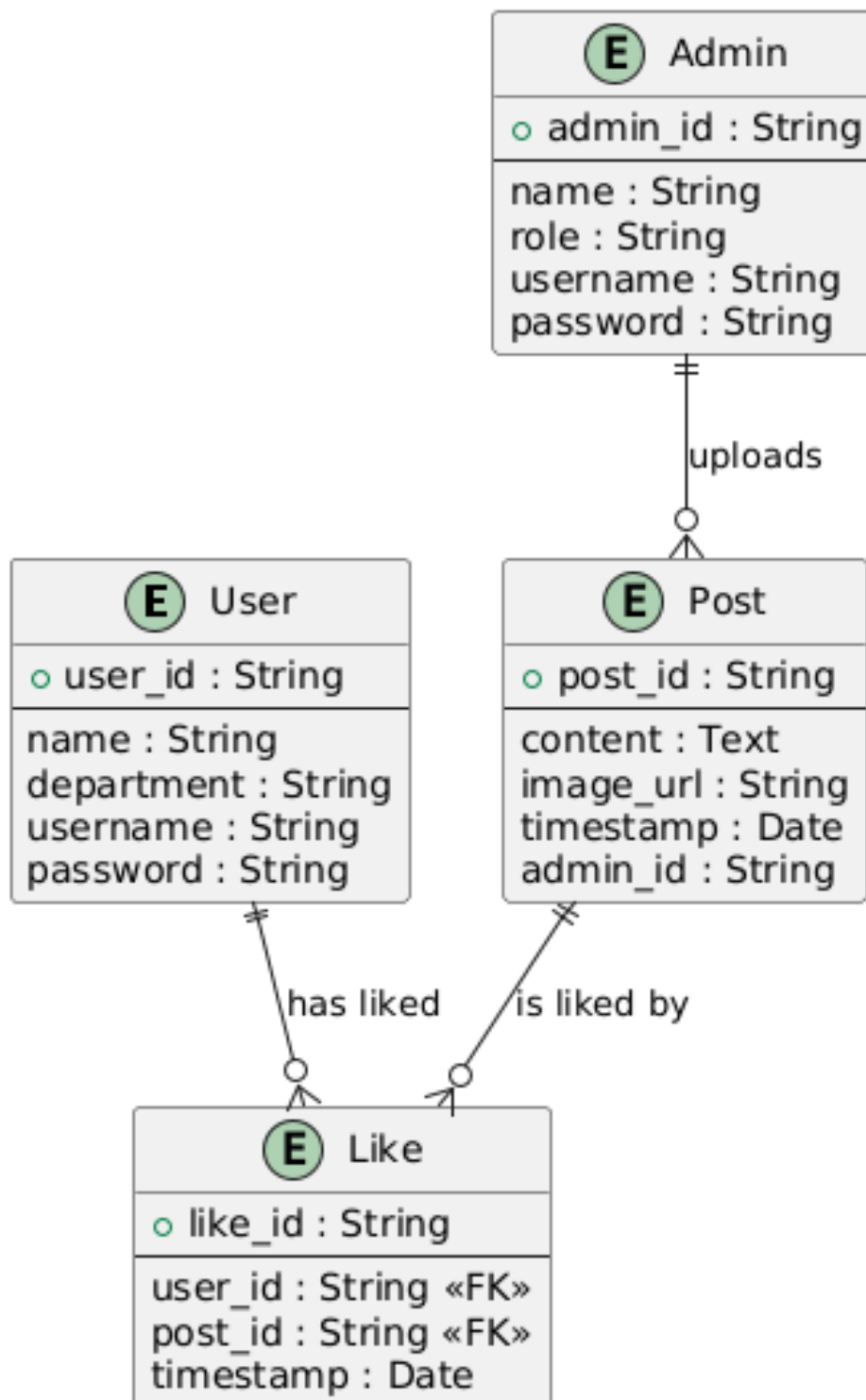
Admin Table (Firebase NoSQL structure)

Field Name	Data Type	Description
name	String	Name of the admin.
email	String	Email address of the admin.
username	String	User Name of the admin.
password	String	Password of the admin.
contact_no	Integer	Phone number or contact details of the admin.

User Table (Firebase NoSQL structure)

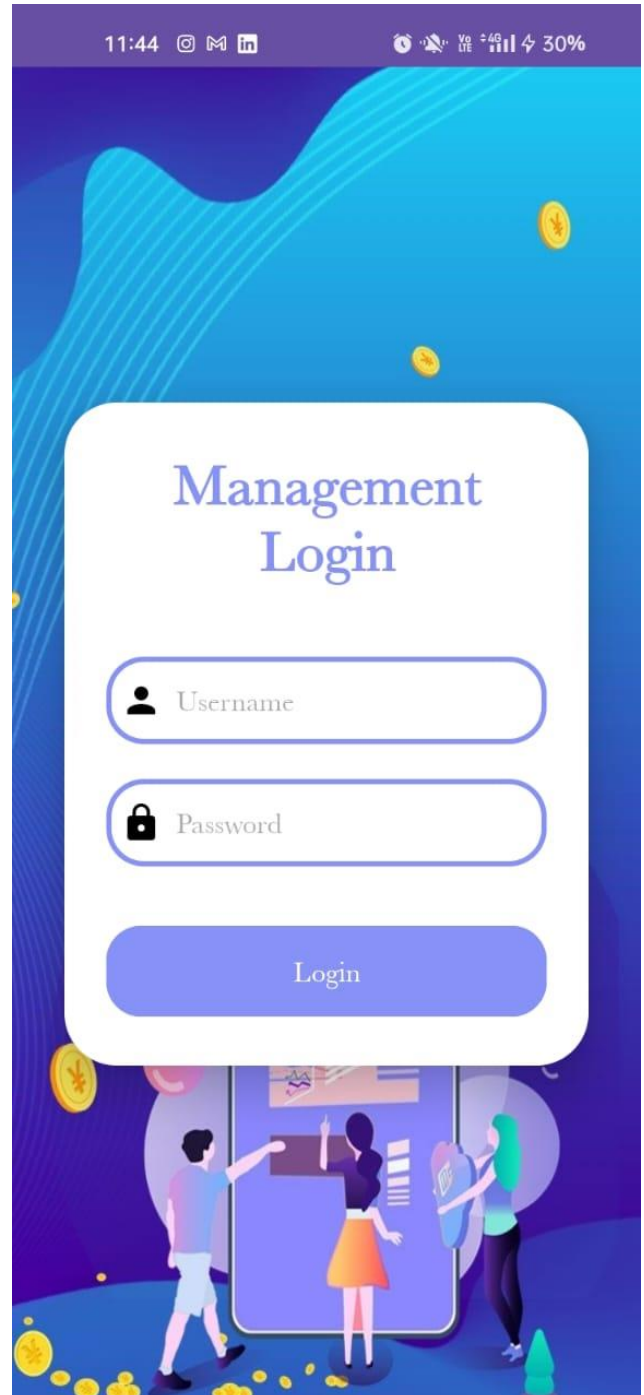
Field Name	Data Type	Description
user_id	String	Unique identifier for each user (e.g., "AU200517").
name	String	Full name of the user (e.g., "Aswin").
department	String	Department to which the user belongs (e.g., "Computer Science").
username	String	Username of the user, which may be the same as user_id.
password	String	Password for the user's account (in plaintext or hashed).

3.3.2 Table Relationship Diagram:



UI DESIGN FOR SHC Social Media:

Login Home & Management Login Module:



Purpose: The LoginHome module serves as the primary access point for admin (management) and users (students) of the SHC Social Media App. It allows admin and students to log in and access the content and functionalities available to them within the application.

Purpose: The AdminLogin module is specifically designed for the management team of the SHC Social Media App. This module allows authorized administrators to log in and manage the content, including creating, editing, or deleting posts.

11:20 [Icons] 30%

 SACRED HEART COLLEGE (Autonomous)
TIRUPATTUR - 635 601, TAMIL NADU.
(Affiliated to Thiruvalluvar University, Vellore.)

Dear Students,

As we approach the end of the **ODD Semester**, I would like to request your **feedback on the courses teachers** of this semester. Your responses will be of great help to the faculty members in enhancing their teaching in the future.

Please note that your feedback is kept **confidential** and it is **mandatory** for every student to provide it. You can give your responses through your login from **5th to 11th October 2024**.

I would appreciate it if you could complete the task on or before the above-mentioned date.

Thank you.

Date: 30/09/2024

 Principal 30/09/24
Principal
Sacred Heart College (Autonomous)
Tirupattur-635 601, Tirupattur - Dist

Phone : 04179-220553 Fax : 04179-220553 Email : office@shcstpt.edu

Course Evaluation Due Date

♥ 11 Likes

Management

Students are asked to clear **the following dues** well before **22.10.2024**. Do not wait to clear the dues till last moment.

Date: 27.09.2024

 Principal 27/09/24
Principal
Sacred Heart College (Autonomous)
Tirupattur-635 601, Tirupattur - Dist

1. College Fee
2. Retreat
3. Book Fee (Tamil, English and Communicative English)
4. Hostel Due & Mess Fees (for Hostel Students only)
5. Certificates Due
6. Student Feedback
7. UMIS
8. Tamil Pudhalvan
9. Puthumal Pen
10. SC, ST & SCC Scholarship Application Form
11. BC, MBC & DNC Scholarship Application Form (Aided mode only)
12. NSP (CSSS - Eligible Students) Scholarship Application Form
13. Other Dues

Due Lists

♥ 7 Likes

Upload Post Delete Post

Purpose: The Management Module is designed to empower the admin team with the tools necessary to manage posts, oversee user interactions, and maintain the integrity of the platform. This module ensures that all content aligns with the app's objectives and meets community standards.

Upload Post Button to upload the new Post on this App.

Delete Post Button to Delete the Existing Post on this App.

Upload Post Module:

11:37



 30%

▼

MCA

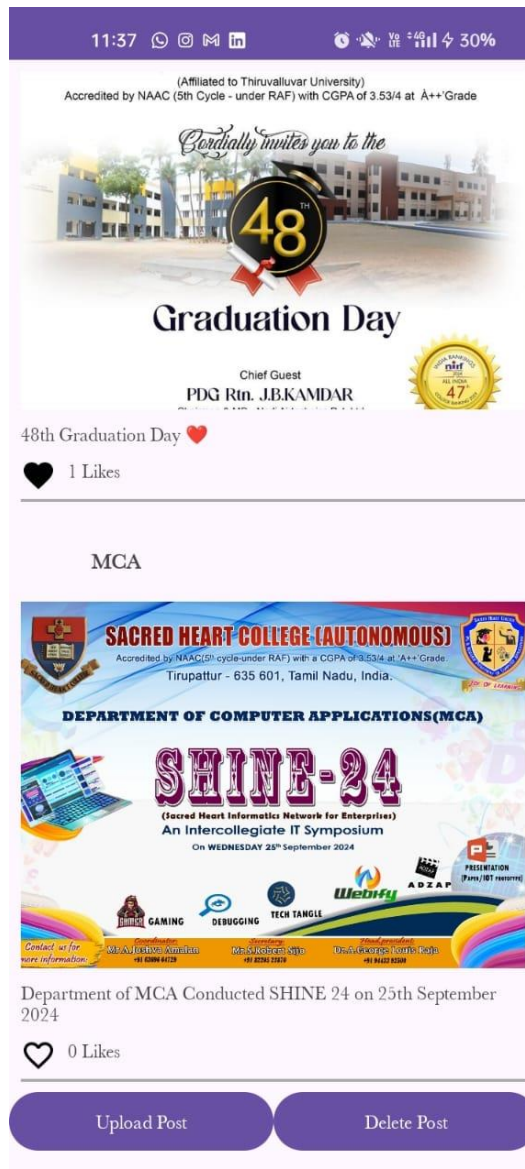


The poster for SHINE-24, an intercollegiate IT symposium organized by the Department of Computer Applications (MCA) at Sacred Heart College (Autonomous). The event took place on Wednesday, 25th September 2024. The poster features the college's logo, accreditation details (NAAC A++ Grade), and various event activities like Gaming, Debating, Tech Tangle, and Webinars. It also lists sponsors and contact information for more details.

Department of MCA Conducted SHINE 24
on 25th September 2024

Choose File

Upload Post



Upload Post

Upload Post feature is to enable the admin to disseminate important information, updates, and announcements to all users within the app. This can include various types of content such as:

- Announcements (e.g., events, deadlines)
- News updates (e.g., changes in policies)
- Educational content (e.g., articles, tips)
- Multimedia content (e.g., images, videos)

1. Initiation:

- The **Upload Post** functionality is initiated when the admin clicks the "**Upload Post**" button within the Admin Module. This action leads to the opening of the upload interface.

2. User Interface Elements:

- **Post Description Field:** A text input field where the admin can write a detailed description of the post. This field may support rich text formatting, allowing for better presentation of the content.
- **Media Upload Option:** An option to upload multimedia files such as images or videos. The admin can click a button (e.g., "**Choose File**") to select files from their device.
- **Department Selection (if applicable):** If the app categorizes posts by departments, there may be a dropdown menu or spinner that allows the admin to select the relevant department associated with the post.

3. File Selection:

- When the admin clicks the "**Choose File**" button, an **Intent** is triggered to allow the admin to select a media file (image or video) from their device. This interaction typically involves:
 - Launching a file picker interface.
 - Filtering for image and video files.
 - Allowing the admin to select a file.

4. Post Submission:

- After filling out the description and selecting the media file, the admin clicks the "**Upload**" button to submit the post. The submission process typically includes the following steps:
- **Input Validation:** The app checks to ensure that the description is not empty and that a media file has been selected. If any fields are incomplete, the app may prompt the admin to fill in the necessary information.
- **Uploading to Storage:** The selected media file is uploaded to Firebase Storage. During this process:
 - A unique identifier (post ID) is generated for the new post.

- The file is stored in a designated folder (e.g., "uploads/") with the unique ID as part of the path.

5. Database Entry:

- Once the media upload is successful, the app retrieves the downloadable URL of the media file. The post's information, including the description, department (if selected), media URL, and any other relevant data, is then saved in the Firebase Realtime Database. The post entry may look like this:

Post post = new Post(selectedDepartmentName, uri.toString(), description, 0, postId);

- Here, uri.toString() represents the URL of the uploaded media, and the other parameters are the relevant details of the post.

6. User Feedback:

- After successfully uploading the post, the admin receives a notification (e.g., a toast message) indicating that the post has been uploaded successfully. This feedback helps confirm that their action has been completed.

7. Error Handling:

- If any errors occur during the upload process (e.g., network issues, invalid input), the app will handle these gracefully by informing the admin of the specific error encountered, allowing them to take corrective actions.

Delete Post Module:



- Remove outdated announcements or posts that are no longer applicable.
- Maintain the integrity and relevance of the content available to users.
- Ensure that inappropriate or erroneous posts can be swiftly dealt with.

1. Initiation:

- The **Delete Post** process begins when the admin selects a post they wish to delete. This can typically be done by:
 - Navigating to a list of existing posts displayed in the admin interface.
 - Long-pressing on a post or selecting a post from a list to reveal additional options, including the delete option.

2. User Interface Elements:

- **Post List View:** The admin is presented with a list or grid of all existing posts, often including brief descriptions, media thumbnails, and metadata (like the date of posting).
- **Delete Button:** Each post in the list will have an associated delete button or icon, allowing the admin to easily initiate the deletion process for that specific post.

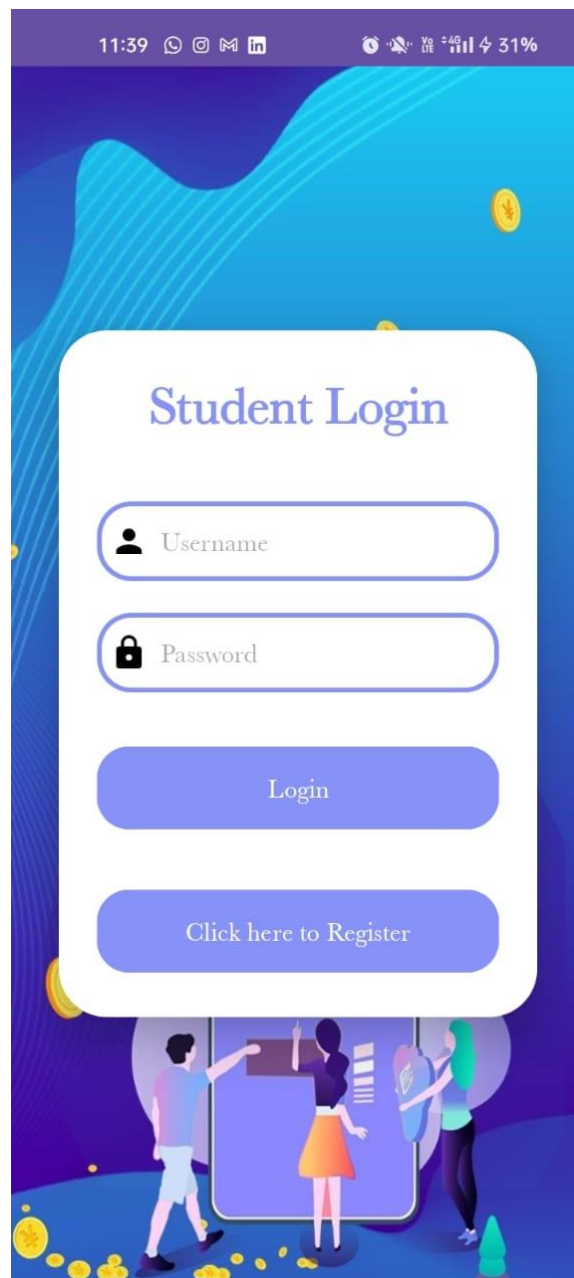
3. Confirmation Dialog:

- Upon clicking the delete option, the app prompts the admin with a confirmation dialog. This dialog typically includes:
 - A message asking the admin to confirm that they wish to delete the selected post.
 - Options for confirming (e.g., "Yes, delete it") or canceling the deletion (e.g., "Cancel").

4. Deletion Process:

- If the admin confirms the deletion:
 - The app will first remove the post from the Firebase Realtime Database using the unique post ID.
 - It may also delete the associated media file from Firebase Storage to free up space and maintain data integrity.

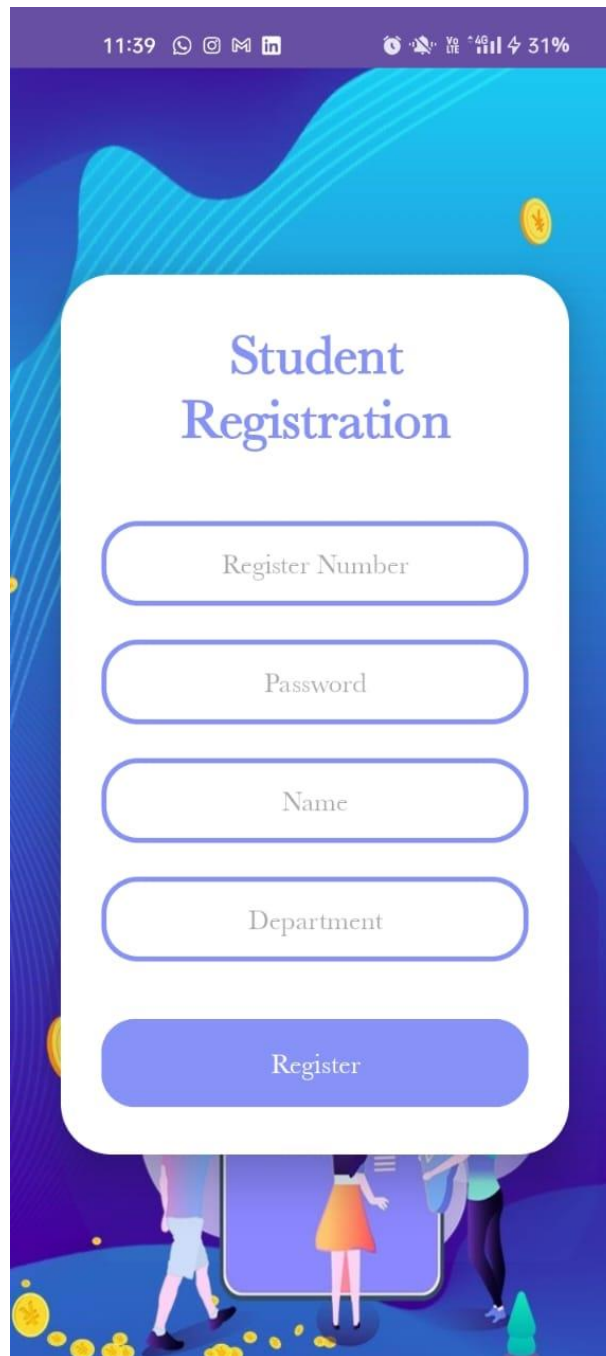
Student Module



The main objectives of the **Student Login** module are to:

- **Authenticate Users:** Ensure that only registered students can access the app and its features, providing a secure environment for sharing information.
- **Personalize Experience:** Allow the app to provide a tailored experience based on the logged-in user, such as displaying relevant posts and notifications.
- **Track User Activity:** Keep track of user interactions, such as posts liked and posts viewed, to enhance engagement and provide a better user experience.

Student Registration



11:39 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000

1. **Gather Student Information:** Collect necessary data to create user accounts for students.
2. **Authenticate New Users:** Ensure that the registration process includes validation checks to prevent duplicate accounts and unauthorized registrations.
3. **Facilitate Access to the App:** Allow new users to create accounts so they can log in and engage with the app's features.

CHAPTER IV

Implement of Java Code:

LoginHome.java

```
package com.example.shcmedia_app;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;

import androidx.appcompat.app.AppCompatActivity;
public class LoginHome extends AppCompatActivity
{

    Button adminbtn, userbtn;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.login_home);

        adminbtn = findViewById(R.id.adminbtn);
        userbtn = findViewById(R.id.userbtn);

        // Set click listener for Admin button
        adminbtn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(LoginHome.this, AdminLogin.class);
                startActivity(intent);
            }
        });

        // Set click listener for User button
        userbtn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(LoginHome.this, UserLogin.class);
                startActivity(intent);
            }
        });
    }
}
```

UploadPost.java

```
package com.example.shcmedia_app;

import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ImageView;
import android.widget.Spinner;
import android.widget.Toast;

import androidx.annotation.Nullable;
import androidx.appcompat.app.AppCompatActivity;

import com.example.shcmedia_app.DepartmentSpinnerAdapter;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.google.firebase.storage.FirebaseStorage;
import com.google.firebase.storage.StorageReference;

import java.util.Arrays;
import java.util.List;

public class UploadPost extends AppCompatActivity {

    private ImageView postImageView;
    private EditText postDescriptionEditText;
    private Button chooseFileButton, uploadPostButton;
    private Uri mediaUri;

    private Spinner departmentLogoSpinner;
    private EditText departmentNameEditText;

    private FirebaseStorage storage;
    private DatabaseReference databaseRef;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.upload_post);
    }
}
```

```

// Initialize UI elements
postImageView = findViewById(R.id.postImageView);
postDescriptionEditText = findViewById(R.id.postDescriptionEditText);
chooseFileButton = findViewById(R.id.chooseButton);
uploadPostButton = findViewById(R.id.uploadPostButton);
departmentLogoSpinner = findViewById(R.id.departmentLogoSpinner);
departmentNameEditText = findViewById(R.id.departmentNameEditText);

// Initialize Firebase
storage = FirebaseStorage.getInstance();
databaseRef = FirebaseDatabase.getInstance().getReference("posts");

// Load the department logos into the Spinner
setupDepartmentSpinner();

// Choose File Button logic
chooseFileButton.setOnClickListener(v -> {
    selectMedia();
});

// Upload Post Button logic
uploadPostButton.setOnClickListener(v -> {
    uploadPost();
});
}

// Method to populate the department logo spinner
private void setupDepartmentSpinner() {
    // Convert String[] to List<String>
    List<String> departmentNames = Arrays.asList("Management", "Computer Science",
"MCA");
    List<String> departmentLogos = Arrays.asList(
        "https://example.com/logo1.png", // Replace with actual URLs or paths
        "https://example.com/logo2.png",
        "https://example.com/logo3.png"
    );

    // Pass lists to the adapter
    DepartmentSpinnerAdapter adapter = new DepartmentSpinnerAdapter(this,
departmentNames, departmentLogos);
    departmentLogoSpinner.setAdapter(adapter);
}

// Method to select image or video
private void selectMedia() {
    Intent intent = new Intent(Intent.ACTION_GET_CONTENT);
    intent.setType("image/* video/*");
}

```

```

        startActivityResult(intent, 101);
    }

    @Override
    protected void onActivityResult(int requestCode, int resultCode, @Nullable Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        if (requestCode == 101 && resultCode == RESULT_OK && data != null) {
            mediaUri = data.getData();
            postImageView.setImageURI(mediaUri); // Show selected image/video preview
            postImageView.setVisibility(View.VISIBLE);
        }
    }

    // Method to upload post to Firebase
    private void uploadPost() {
        if (mediaUri != null && !postDescriptionEditText.getText().toString().isEmpty()) {
            String postId = databaseRef.push().getKey();
            StorageReference storageRef = storage.getReference().child("uploads/" + postId);

            storageRef.putFile(mediaUri).addOnSuccessListener(taskSnapshot -> {
                storageRef.getDownloadUrl().addOnSuccessListener(uri -> {
                    // Get selected department name and logo
                    int selectedLogoPosition = departmentLogoSpinner.getSelectedItemPosition();
                    String selectedDepartmentName = departmentNameEditText.getText().toString();

                    // Save post data in the Realtime Database
                    String description = postDescriptionEditText.getText().toString();
                    Post post = new Post(selectedDepartmentName, uri.toString(), description, 0, postId,
"timestamp", false);
                    databaseRef.child(postId).setValue(post);
                    Toast.makeText(UploadPost.this, "Post Uploaded", Toast.LENGTH_SHORT).show();
                    finish(); // Close the activity
                });
            }).addOnFailureListener(e -> {
                Toast.makeText(UploadPost.this, "Failed to upload: " + e.getMessage(),
Toast.LENGTH_SHORT).show();
                e.printStackTrace(); // Log the error
            });

        } else {
            Toast.makeText(this, "Select a file and add description", Toast.LENGTH_SHORT).show();
        }
    }
}

```

PostAdapter.java

```
package com.example.shcmedia_app;

import android.content.Context;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.ImageView;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.bumptech.glide.Glide;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;

import java.util.List;

public class PostAdapter extends RecyclerView.Adapter<PostAdapter.PostViewHolder> {

    private Context context;
    private List<Post> postList;

    public PostAdapter(Context context, List<Post> postList) {
        this.context = context;
        this.postList = postList;
    }

    @NonNull
    @Override
    public PostViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(context).inflate(R.layout.post_item, parent, false);
        return new PostViewHolder(view);
    }

    @Override
    public void onBindViewHolder(@NonNull PostViewHolder holder, int position) {
        Post post = postList.get(position);

        // Set the post description and department name (username)
        holder.description.setText(post.getDescription());
        holder.username.setText(post.getUsername());
    }
}
```



```

// Load department logo and post image using Glide
Glide.with(context).load(post.getDepartmentLogo()).into(holder.departmentLogo);
Glide.with(context).load(post.getMediaUrl()).into(holder.postImage);

// Set the like count
holder.likeCountTextView.setText(post.getLikes() + " Likes");

// Set the initial like button icon based on whether the post is liked
if (post.isLiked()) {
    holder.likeButton.setImageResource(R.drawable.ic_like); // Already liked
} else {
    holder.likeButton.setImageResource(R.drawable.ic_liked); // Not liked yet
}

// Handle like button clicks
holder.likeButton.setOnClickListener(v -> {
    // Toggle the like status
    boolean isLiked = post.isLiked();
    if (isLiked) {
        post.setLiked(false);
        post.setLikes(post.getLikes() - 1);
        holder.likeButton.setImageResource(R.drawable.ic_liked); // Change to 'not liked' icon
    } else {
        post.setLiked(true);
        post.setLikes(post.getLikes() + 1);
        holder.likeButton.setImageResource(R.drawable.ic_like); // Change to 'liked' icon
    }

    // Update the like count text
    holder.likeCountTextView.setText(post.getLikes() + " Likes");

    // Update Firebase with the new like status and count
    DatabaseReference postRef =
    FirebaseDatabase.getInstance().getReference("posts").child(post.getPostId());
    postRef.child("likes").setValue(post.getLikes());
    postRef.child("liked").setValue(post.isLiked()); // Save the like status in the database
    (optional)
    });
}

@Override
public int getItemCount() {
    return postList.size();
}

public static class PostViewHolder extends RecyclerView.ViewHolder {
    TextView username, description, likeCountTextView;

```

```

        ImageView postImage, likeButton, departmentLogo;

        public PostViewHolder(@NonNull View itemView) {
            super(itemView);
            username = itemView.findViewById(R.id.usernameTextView);
            description = itemView.findViewById(R.id.descriptionTextView);
            postImage = itemView.findViewById(R.id.postImageView);
            likeButton = itemView.findViewById(R.id.likeButton);
            likeCountTextView = itemView.findViewById(R.id.likeCountTextView);
            departmentLogo = itemView.findViewById(R.id.departmentLogoView);
        }
    }
}

```

MainActivity.java

```

package com.example.shcmedia_app;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ImageButton;
import android.widget.TextView;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.firebase.auth.FirebaseAuth;
import com.google.firebase.auth.FirebaseUser;
import com.google.firebase.database.DataSnapshot;
import com.google.firebase.database.DatabaseError;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.google.firebase.database.ValueEventListener;

import java.util.ArrayList;
import java.util.List;

public class MainActivity extends AppCompatActivity {

```

```

private boolean isAdmin;
private RecyclerView postRecyclerView;
private Button uploadPostButton, viewProfileButton, viewLikesButton;
private View adminOptionsLayout;
private TextView welcomeTextView;
private ImageButton logout;

private PostAdapter postAdapter;
private List<Post> postList;
private FirebaseDatabase database;
private DatabaseReference postsRef;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main); // Set the activity_main layout

    // Initialize RecyclerView
    postRecyclerView = findViewById(R.id.postRecyclerView);
    postRecyclerView.setLayoutManager(new LinearLayoutManager(this));

    postList = new ArrayList<>();
    postAdapter = new PostAdapter(this, postList);
    postRecyclerView.setAdapter(postAdapter);

    database = FirebaseDatabase.getInstance();
    postsRef = database.getReference("posts");

    // Retrieve the "isAdmin" flag and username from the Intent
    Intent intent = getIntent();
    isAdmin = intent.getBooleanExtra("isAdmin", false); // Fetch whether the user is an admin
    String username = intent.getStringExtra("username"); // Fetch the username from the Intent

    // Bind views
    // Bind the welcome TextView
    uploadPostButton = findViewById(R.id.uploadPostButton);
    viewProfileButton = findViewById(R.id.viewProfileButton);
    adminOptionsLayout = findViewById(R.id.adminOptionsLayout); // Admin buttons
    container layout

    // Set the welcome message with the username
    if (username != null && !username.isEmpty()) {

```

```

        welcomeTextView.setText("Welcome, " + username + "!");
    }

    // Toggle visibility of admin-specific buttons based on isAdmin flag
    if (isAdmin) {
        adminOptionsLayout.setVisibility(View.VISIBLE); // Show admin options for admins
    } else {
        adminOptionsLayout.setVisibility(View.GONE); // Hide admin options for regular users
    }

    // Set up button click listeners if user is an admin
    if (isAdmin) {
        // Handle the upload post button click
        uploadPostButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent uploadIntent = new Intent(MainActivity.this, UploadPost.class);
                startActivity(uploadIntent);
            }
        });

        // Handle the view profile button click
        viewProfileButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                // Navigate to Profile page (modify if Profile page is a different activity)
                Intent profileIntent = new Intent(MainActivity.this, PostListActivity.class);
                startActivity(profileIntent);
            }
        });

        // Handle the view likes button click
    }

    // Load posts from Firebase
    loadPosts();
}

private void loadPosts() {
    postsRef.addValueEventListener(new ValueEventListener() {
        @Override
        public void onDataChange(@NonNull DataSnapshot snapshot) {
            postList.clear();
            for (DataSnapshot postSnapshot : snapshot.getChildren()) {
                Post post = postSnapshot.getValue(Post.class);
            }
        }
    });
}

```

```

        postList.add(post);
    }
    postAdapter.notifyDataSetChanged();
}

@Override
public void onCancelled(@NonNull DatabaseError error) {
    Toast.makeText(MainActivity.this, "Failed to load posts",
Toast.LENGTH_SHORT).show();
}
});
}
}

```

UserRegistration.java

```

package com.example.shcmedia_app;

import android.content.Intent;
import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;

public class UserRegister extends AppCompatActivity {

    // Declare EditText and Button variables
    private EditText editTextRegNumber, editTextPassword, editTextName, editTextDept;
    private Button buttonRegister;

    // Firebase Database reference
    private DatabaseReference databaseReference;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.user_register); // Set the layout file for User Registration

        // Initialize Firebase Database reference
    }
}

```

```

databaseReference = FirebaseDatabase.getInstance().getReference("user");

// Bind the layout components with the variables
editTextRegNumber = findViewById(R.id.usernameReg);
editTextPassword = findViewById(R.id.userpasswordReg);
editTextName = findViewById(R.id.usernameReg);
editTextDept = findViewById(R.id.userdeptReg);
buttonRegister = findViewById(R.id.loginStaffButton);

// Set onClickListener for the Register button
buttonRegister.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        // Get the input values
        String regNumber = editTextRegNumber.getText().toString().trim();
        String password = editTextPassword.getText().toString().trim();
        String name = editTextName.getText().toString().trim();
        String department = editTextDept.getText().toString().trim();

        // Validate input fields
        if (TextUtils.isEmpty(regNumber) || TextUtils.isEmpty(password) ||
            TextUtils.isEmpty(name) || TextUtils.isEmpty(department)) {
            Toast.makeText(UserRegister.this, "Please fill all fields",
                Toast.LENGTH_SHORT).show();
        } else {
            // Register user and store data in Firebase
            registerUser(regNumber, password, name, department);
        }
    }
});

// Method to register user and store data in Firebase
private void registerUser(String regNumber, String password, String name, String department)
{
    // Create a unique key for each user (you can use the regNumber as the key)
    DatabaseReference userRef = databaseReference.child(regNumber);

    // Create a User object to store the data
    User user = new User(name, department, password, regNumber);

    // Store the user data in the Firebase database under the unique key
    userRef.setValue(user).addOnCompleteListener(task -> {
        if (task.isSuccessful()) {
            Toast.makeText(UserRegister.this, "Registration Successful",
                Toast.LENGTH_SHORT).show();
            // Clear input fields

```

```

        clearFields();
        // Navigate to UserLogin activity
        Intent intent = new Intent(UserRegister.this, UserLogin.class);
        startActivity(intent);
    } else {
        Toast.makeText(UserRegister.this, "Registration Failed",
Toast.LENGTH_SHORT).show();
    }
});
}

// Method to clear input fields after registration
private void clearFields() {
    editTextRegNumber.setText("");
    editTextPassword.setText("");
    editTextName.setText("");
    editTextDept.setText("");
}

// User class to represent the user data structure
public static class User {
    public String name;
    public String department;
    public String password;
    public String username;

    public User() {
        // Default constructor required for calls to DataSnapshot.getValue(User.class)
    }

    public User(String name, String department, String password, String username) {
        this.name = name;
        this.department = department;
        this.password = password;
        this.username = username;
    }
}
}

```

CHAPTER V

Testing for SHC Social Media

The purpose of this test plan is to define the testing approach for the SHC Social Media App. The app is designed for students to view posts, like them, and for admins to upload and delete posts. This test plan focuses on ensuring all functionalities work as expected and the app performs smoothly across different scenarios.

Scope of Testing:

Test Case ID	Test Description	Expected Result	Status
TC_001	Verify admin login with valid credentials	Admin should be able to log in successfully	Pass
TC_002	Verify post upload with valid media and description	Post should be uploaded and visible to all students	Pass
TC_003	Verify post deletion by admin	Selected post should be deleted and removed from student view	Pass
TC_004	Verify student registration with valid details	Student should be able to register and log in	Pass
TC_005	Verify like functionality	Post's like count should increase when a student clicks the like button	Pass

Admin:

The **Admin Module** allows the admin (Management team) to upload posts containing important updates, delete posts, and manage the app's content. Testing for this module will focus on the functionality, usability, and security of the admin features to ensure smooth management of the platform.

Field Name	Data Type	Constraints	Description
admin_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each admin user.
email	VARCHAR(255)	UNIQUE, NOT NULL	Admin email used for login.
password	VARCHAR(255)	NOT NULL	Hashed password for security.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp for when the admin account was created.
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Timestamp for the last update.

Posts

The Posts Table in the SHC Social Media App is designed to store and manage all the posts uploaded by the admin. These posts are intended for circulation of management-related information, such as announcements and updates, that students can view and like.

Field Name	Data Type	Constraints	Description
post_id	STRING	PRIMARY KEY	Unique identifier for each post.
department_name	VARCHAR(100)	NOT NULL	Name of the department associated with the post.
media_url	VARCHAR(255)	NOT NULL	URL of the media file (image or video).
description	TEXT	NOT NULL	Text description of the post.
likes	INT	DEFAULT 0	Number of likes the post has received.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp for when the post was created.
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Timestamp for the last update.

5.1 Test Plan

Test Case ID	Test Scenario	Expected Result	Status
TC_ADMIN_01	Admin login and authentication	Admin can log in with valid credentials and access the admin dashboard.	Pass
TC_ADMIN_02	Upload a post	Admin can successfully upload a post with an image or video and description.	Pass
TC_ADMIN_03	View all posts in the admin panel	Admin can view all uploaded posts in the admin panel.	Pass
TC_ADMIN_04	Delete a post	Admin can delete posts successfully from the list after long-pressing on them.	Pass
TC_ADMIN_05	View upload status	Admin can view the upload status of each post (e.g., uploaded, in progress).	Pass
TC_ADMIN_06	Handle media upload failure	Admin is notified of media upload failure and given options to retry or cancel.	Pass
TC_STUDENT_01	Student login and authentication	Students can log in with correct credentials to access the app.	Pass
TC_STUDENT_02	View posts on the homepage	Students can view posts uploaded by the admin on the homepage.	Pass
TC_STUDENT_03	Like a post	Students can like a post, and the like count is updated accordingly.	Pass
TC_STUDENT_04	View post details	Students can tap a post to view its details (media and description).	Pass
TC_STUDENT_05	View like status	Students can see if they have liked a post and the total number of likes.	Pass
TC_STUDENT_06	Media loading for posts	Students can see media (image or video) of posts without delays or errors.	Pass
TC_STUDENT_07	Handle login errors	Students receive appropriate error messages for incorrect login attempts.	Pass
TC_STUDENT_08	Secure student data	Ensure student information is securely stored and accessible only to the user.	Pass
TC_ADMIN_07	Data consistency across devices	Posts uploaded by admin are visible across student devices immediately.	Pass
TC_ADMIN_08	Error handling on invalid actions	Admin is shown error messages when trying invalid actions (e.g., delete a non-existent post).	Pass

TC_STUDENT_09	Handle no internet connection	Students are informed of network issues when trying to view posts without internet.	Pass
TC_STUDENT_10	App layout and navigation	Students can easily navigate the app without issues, with all buttons functioning.	Pass

Scope of Testing

- **Admin Module:** Focuses on post creation, deletion, and viewing functionalities.
- **Student Module:** Focuses on post viewing, liking, and navigation through the app.

This structure tests both admin and student functionalities in the SHC Social Media App and ensures that posts are correctly managed by the admin and viewed/interacted with by the students.

5.2 Test Cases

1. Unit Testing

- **Objective:** Test individual components and functions of the SHC Social Media App to ensure they work correctly in isolation.
- **Strategy:**
 - Focus on testing critical functions like admin post uploading, student login, viewing posts, and liking functionality.
 - Testing for each function in isolation, such as validating correct post upload to Firebase, checking if like counts update correctly, and verifying media display.
 - Tools like JUnit for testing backend logic and Mockito for mocking Firebase and other external dependencies.
 - Example: Test the method that uploads posts to Firebase and the method that handles the deletion of posts.

2. Integration Testing

- **Objective:** Ensure that different modules (admin and student) work together smoothly.
- **Strategy:**
 - Test the interaction between different modules (e.g., integration between post upload, post view, and like features).
 - Verify that data flows correctly from admin post creation to student viewing and liking.
 - Test cases to cover key integrations like post uploading, deletion, and real-time updates of likes.
 - Example: Verify that after an admin deletes a post, it no longer appears in the student module and that the Firebase database is updated accordingly.

3. System Testing

- **Objective:** Test the entire SHC Social Media App system to validate that it functions according to the requirements.
- **Strategy:**
 - Conduct end-to-end testing, including admin login, post creation, student viewing, and liking posts.
 - Ensure all features and workflows function as expected (e.g., admin can upload posts, and students can interact with those posts by liking them).
 - Test scenarios like multiple users interacting with the app simultaneously, ensuring performance remains stable.
 - Example: Perform a complete test from admin uploading a post to students viewing and liking the post across multiple devices.

4. User Interface (UI) Testing

- a. **Objective:** Ensure the app's user interface is intuitive, responsive, and consistent across different devices and screen sizes.
- b. **Strategy:**
 - i. Verify that UI elements like buttons, forms, and menus function correctly across a variety of Android devices.
 - ii. Test the responsiveness of the app on different screen sizes and Android versions, ensuring that layout and navigation remain consistent.
 - iii. Ensure that labels, icons, and other UI elements are aligned and user-friendly.
 - iv. Example: Test the post upload form on different screen resolutions to ensure that it scales correctly and the user experience remains intact.

5. Functional Testing

- a. **Objective:** Ensure that the SHC Social Media App meets all the functional requirements defined in the project documentation.
- b. **Strategy:**
 - i. Test core functionalities like admin login, post upload, student viewing, and post interaction (liking posts).
 - ii. Ensure students can log in, view posts, and like posts, and that admins can upload and delete posts successfully.

- iii. Validate that the media upload and download features work correctly across various devices.
- iv. Example: Test whether a student's "like" on a post updates in real-time and whether the admin can successfully delete posts.

6. Regression Testing

- a. **Objective:** Ensure that new changes or updates in the SHC Social Media App do not negatively impact existing functionality.
- b. **Strategy:**
 - i. Create a regression test suite covering key functionalities like post upload, deletion, and student interaction with posts.
 - ii. Re-run tests whenever new features are added (like post categorization or additional media types) or when existing ones are modified.
 - iii. Focus on key workflows like post management by the admin and real-time post interactions for students.
 - iv. Example: After implementing new features such as post editing, verify that the upload and deletion features still work as intended.

7. Performance Testing

- a. **Objective:** Ensure that the SHC Social Media App performs efficiently under different loads and stress conditions.
- b. **Strategy:**
 - i. Test the app's performance when multiple students are logged in and interacting with posts simultaneously.
 - ii. Check response times when uploading posts, liking posts, and viewing media content.
 - iii. Conduct load testing to ensure the app can handle a high number of concurrent users without performance degradation.
 - iv. Example: Simulate 50 students liking posts simultaneously and check the app's response and performance.

8. Security Testing

- a. **Objective:** Identify potential vulnerabilities in the SHC Social Media App and ensure that it is secure.
- b. **Strategy:**
 - i. Test for potential security risks like data leakage, unauthorized access, and improper session handling.

- ii. Ensure that student and admin data (like login credentials) are securely stored and transmitted via secure protocols.
- iii. Validate proper authentication mechanisms for admins and students to ensure only authorized actions (like post deletion) are allowed.
- iv. Example: Test that admin credentials cannot be compromised, and students cannot access or modify admin-level functionalities.

9. Acceptance Testing

- a. **Objective:** Ensure that the SHC Social Media App meets the business requirements and satisfies the needs of the end users (admins and students).
- b. **Strategy:**
 - i. Conduct user acceptance testing (UAT) with the admin and a group of students to gather feedback on the usability and performance of the app.
 - ii. Validate that the app's features and workflows align with the initial project requirements and goals.
 - iii. Gather feedback to identify areas for improvement, such as UI design or workflow efficiency.
 - iv. Example: Conduct a live UAT session where the admin uploads posts, and students interact with those posts, verifying that all functionalities work seamlessly.

10. Compatibility Testing

- a. **Objective:** Ensure that the SHC Social Media App works across different Android devices, screen resolutions, and OS versions.
- b. **Strategy:**
 - i. Test the app on various Android devices, ensuring it runs smoothly across different screen sizes and OS versions.
 - ii. Ensure the app's UI, media handling, and performance are consistent across all supported devices.
 - iii. Verify that functionalities like post upload, media display, and navigation remain consistent across devices.
 - iv. Example: Test the app on devices with Android versions 8.0, 10.0, and 11.0, ensuring that the post upload and viewing features behave consistently.

5.3 Test Report

1. Android Studio

- **Purpose:** Used for functional, performance, and unit testing of the SHC Social Media App.
- **Usage:**
 - Provides the environment for writing, testing, and debugging the app.
 - Includes built-in testing frameworks like **JUnit** for unit testing the app's backend logic and **Espresso** for UI testing, such as validating admin post uploads and student interactions with posts (e.g., liking posts).
 - Supports continuous testing during the development lifecycle to ensure that both admin and student functionalities work as intended.

2. Firebase Console

- **Purpose:** To validate database entries and ensure data is stored correctly in **Firebase Realtime Database**.
- **Usage:**
 - Monitors and manages Firebase data for user posts, likes, and admin activities, ensuring that all actions are correctly recorded and retrieved.
 - Provides real-time feedback on how data flows between the admin (for post creation/deletion) and student modules (for viewing/liking posts).
 - Helps verify that the correct data (e.g., post details, like counts) is being written to and read from the Firebase database during various operations.

3. Manual Testing

- **Purpose:** For **User Acceptance Testing (UAT)** to assess usability and functionality.
- **Usage:**
 - Involves real-world testing of the application with actual end-users (admins and students) to verify the app's overall usability, responsiveness, and whether it meets the project's business goals.

- Includes testing scenarios like the admin uploading posts, students viewing posts, and interacting through the "like" feature to ensure the system behaves as expected.
- Checks that all core functionalities, like post management and media display, align with user requirements.

4. **Firebase Test Lab**

- **Purpose:** Cloud-based testing on real Android devices.
- **Usage:**
 - Runs automated tests on various real-world Android devices to verify compatibility, performance, and app stability across different devices.
 - Ensures that the app's UI and functionality, such as post uploads by the admin and post interaction by students, remain consistent and error-free on multiple device types and OS versions.

5. **Crashlytics (Firebase)**

- **Purpose:** For crash reporting and app stability tracking.
- **Usage:**
 - Identifies and logs any app crashes during testing phases, particularly when students are interacting with posts or the admin is managing content.
 - Helps developers track issues like app crashes or performance bottlenecks, ensuring a more stable release by fixing bugs before deployment.
 - Useful for monitoring app stability when handling large numbers of simultaneous user interactions, like multiple students liking posts at the same time.

6. **Appium**

- **Purpose:** For mobile automation testing.

- **Usage:**

- Automates functional testing to ensure that core actions like admin post uploads, post deletion, and student interactions (viewing and liking posts) work correctly on mobile devices.
- Supports testing of app functionality across different Android devices, automating the testing process to improve efficiency and ensure broad compatibility.

Test Conclusion:

The testing process for the **SHC Social Media App** was comprehensive, covering various aspects such as functionality, integration, security, and user experience. Through **unit testing**, individual components like user authentication and post management were validated, ensuring they work as expected in isolation. **Integration testing** confirmed the seamless interaction between modules such as the admin's ability to upload posts and students' ability to view and like those posts.

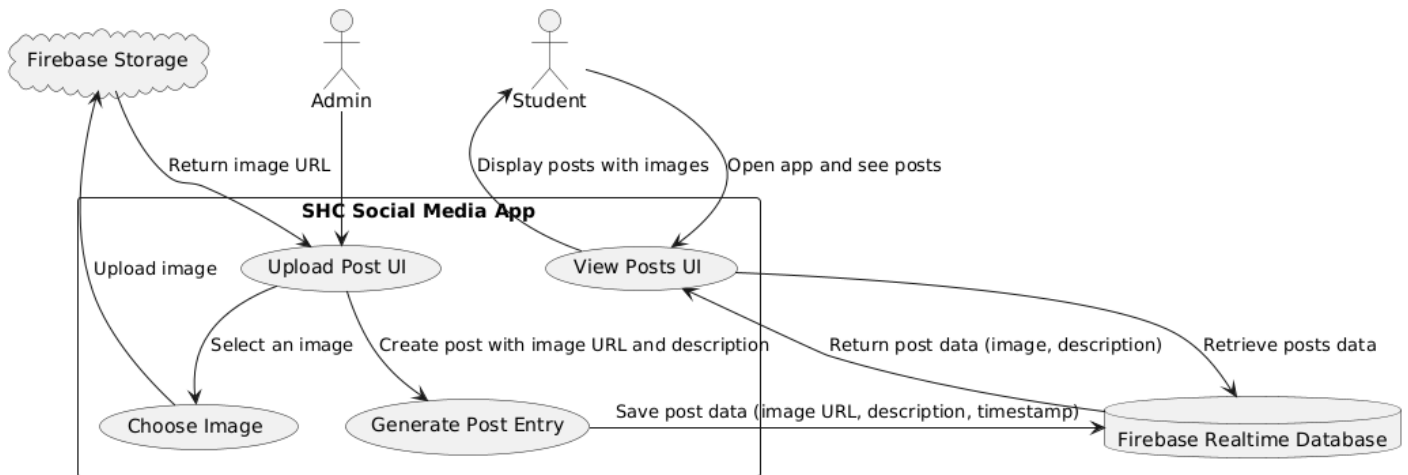
System testing verified the entire app workflow, from admin post management to student engagement, ensuring all features aligned with the project requirements. **Performance testing** showed that the app can handle multiple users and maintain responsiveness even under load, while **security testing** ensured that sensitive data such as admin credentials and post information are secure.

Real-world scenarios were also validated through **user acceptance testing (UAT)**, ensuring that both the admin and students found the app easy to use and effective in its purpose. **Manual testing** provided additional feedback on the overall usability and responsiveness of the app.

In conclusion, the SHC Social Media App passed all critical tests, confirming that it meets the desired functionality, is secure, performs well under different conditions, and delivers a smooth user experience for both the admin and student roles.

CHAPTER VI

Implementation and Deployment



1. Admin Role:

- The admin has the ability to upload posts that include an image and a description.
- The admin workflow:
 - **Choose Image:** Admin selects an image from their device.
 - **Upload Post UI:** The app uploads the selected image to **Firestore Storage**.
 - Once the image is uploaded, Firestore Storage returns the image's URL.
 - **Generate Post Entry:** A post entry is created using the image URL, a description, and a timestamp.
 - The post data (image URL, description, and timestamp) is then saved to **Firestore Realtime Database**.

2. Student/User Role:

- Students/users can only view the posts uploaded by the admin.
- The student workflow:
 - **View Posts UI:** The app retrieves the posts (image URL, description, and timestamp) from **Firestore Realtime Database**.
 - The app displays the posts with their associated images and descriptions for students to view.

Firestore Components:

1. Firestore Storage:

- This service stores the images uploaded by the admin.
- When an image is uploaded, Firestore Storage provides a URL for accessing the image.

2. Firestore Realtime Database:

- This service stores the post data, including the image URL, description, and timestamp.
- It allows the app to retrieve posts in real-time and display them in the **View Posts UI**.

Workflow Summary:

• Admin:

- Selects an image → Image is uploaded to Firestore Storage → Image URL is returned.
- Creates a post with image URL and description → Post data is stored in Firestore Realtime Database.

• Student:

- Opens the app → Post data (including image URLs and descriptions) is retrieved from Firestore Realtime Database → Posts are displayed.

Implementation and Deployment Considerations:

1. Firestore Setup:

- **Firestore Storage** is used to upload and store images.
- **Firestore Realtime Database** is used to store metadata about each post (image URL, description, timestamp).

2. UI Design:

- Admin has access to an **Upload Post UI** for uploading posts.
- Students have access to the **View Posts UI** where posts are displayed.

3. Security:

- Ensure that Firestore security rules are properly configured so that only the admin can upload posts, while students can only view them.

4. Real-Time Data:

- Firestore Realtime Database ensures that the posts are displayed in real-time to students whenever new content is added by the admin.

CHAPTER VII

7.1 I/O Report

1. Admin - Upload Post Module

Input:

- Image or video file (chosen by the admin)
- Post description (text)
- Post title (text)
- Department name (optional input field)

Output:

- Post is saved in Firebase Realtime Database with a unique post ID.
- Image/video file is uploaded to Firebase Storage.
- URL of the uploaded media is saved in the post data.
- Success message displayed to the admin ("Post Uploaded Successfully").
- The post is visible to students in the post list.

2. Student - View Posts Module

Input:

- Request to view posts when opening the app or refreshing the feed.

Output:

- Posts (title, description, media URL) retrieved from Firebase Realtime Database.
- Images and videos are fetched from Firebase Storage based on the media URL.
- Posts are displayed on the user interface (UI) for students to view.
- Students can see the number of likes on each post.

3. Student - Like Post Module

Input:

- Student selects a post to like by clicking the "Like" button.

Output:

- The like count of the post is incremented in the Firebase Realtime Database.
- The updated like count is displayed to all students viewing the post.
- Confirmation message displayed to the student ("Liked").

4. Admin - Delete Post Module

Input:

- Admin selects a post to delete from the post list (usually by long-pressing or clicking a "Delete" button).

Output:

- Post is removed from Firebase Realtime Database.
- Associated media file (image or video) is deleted from Firebase Storage.
- Success message displayed to the admin ("Post Deleted Successfully").
- Post is no longer visible to students in the post list.

5. System - Data Synchronization

Input:

- Changes made to the post database (e.g., new post added, post deleted, like count updated).

Output:

- Real-time synchronization of post data across all students' devices using Firebase Realtime Database.
- Immediate reflection of the latest posts and updates (likes, deletions) on the students' feed.

Module	Input	Output
Admin - Upload Post	Image/Video, Title, Description	Post uploaded to Firebase, visible to students, success message shown
Student - View Posts	Request to view posts	Posts retrieved from Firebase, media loaded from storage, posts displayed in UI
Student - Like Post	Click on "Like" button	Like count incremented, updated like count displayed to all users
Admin - Delete Post	Select post to delete	Post removed from Firebase, media file deleted, success message shown, post no longer visible to users
System - Synchronization	Any data changes (e.g., new post, delete, like)	Real-time updates reflected in all students' UI

7.2 Conclusion:

The **SHC Social Media App** successfully achieves its core objective of creating a dedicated platform for sharing management-related information among students. Through this platform, the admin team can effortlessly upload posts, including images and videos, while students can view, like, and interact with these posts.

The app's use of **Firebase** for real-time database management and cloud storage ensures seamless communication, synchronization, and data handling across all users. The implementation of essential features like **post uploads, likes, and deletions** guarantees smooth content management and provides students with up-to-date information in a user-friendly format.

Key highlights of the project include:

- **User-friendly Interface:** The app offers an intuitive interface for both admins and students, ensuring easy access to posts and seamless navigation.
- **Efficient Content Management:** Admins can upload and delete posts efficiently, allowing for quick dissemination of information.
- **Real-time Interaction:** Students can interact with the posts by liking them, and all changes are reflected instantly across all devices thanks to Firebase's real-time synchronization capabilities.
- **Secure and Scalable:** By leveraging Firebase for backend operations, the app ensures data security and can scale effortlessly as the number of users increases.

In conclusion, the SHC Social Media App provides a modern, efficient, and real-time communication platform that meets the institution's needs for circulating important management updates and information among the student body. The app is flexible, secure, and adaptable for future enhancements, making it a robust tool for educational communication and engagement.

7.3 Reference:

Android Development:

- URL: <https://developer.android.com/docs> For understanding Android-specific concepts, activities, intents, adapters, and UI components.
- URL: <https://material.io/design> To follow modern UI/UX design principles for Android, focusing on clean and user-friendly interfaces.

Firestore Integration:

- URL: <https://firebase.google.com/docs/database> Used for setting up Firestore Realtime Database to handle post storage, updates, and deletions.

Firestore Authentication Documentation:

- URL: <https://firebase.google.com/docs/auth> For managing admin login functionality and user authentication.

RecyclerView and Adapters:

- URL: <https://developer.android.com/guide/topics/ui/layout/recyclerview> To understand how to use RecyclerView for listing posts and implementing actions like long-press to delete posts.

AndroidX RecyclerView:

- URL: <https://developer.android.com/jetpack/androidx/releases/recyclerview> As the core library for managing lists and viewholders.